---

1. 🎉 Getting Started

- **Sign up & Obtain API Keys**: Create an account and retrieve both *test* and *live* API keys from your dashboard (developer.paychangu.com).

- **Test Mode Setup**: Use test credentials (e.g. card number `4242 4242 4242 4242`, mobile money numbers like `990000000`) to simulate payments and debug before going live (developer.paychangu.com).

---

2. Integration Options

A. Standard Checkout

- Use a server-side API call to generate a hosted payment **link**, redirect customers, and handle redirection upon completion (developer.paychangu.com).

B. Inline Checkout

- Integrate a JavaScript SDK (`popup.js`) to embed a checkout popup directly on your page.

- You call the `PaychanguCheckout()` function with config (public key, amount, currency, tx_ref, customer details, callback/return URLs, etc.) (developer.paychangu.com).

- Post-payment: users are redirected; webhooks are triggered; email receipts sent.

C. Direct Charges

For custom UI flows without popup/redirect, PayChangu supports:

- **Mobile Money**: Send customer's mobile number, start payment; user authorizes with provider; webhook notification on completion (developer.paychangu.com).

- **Bank Transfer**: Generate an account for bank deposit (MWK only), then confirm payment via webhook (developer.paychangu.com).

---

3. Webhooks

- Set up a webhook URL in the dashboard (Settings → API & Webhooks).
- Webhooks send POST notifications for events (e.g., `api.charge.payment`, `api.payout`) with full JSON payload — use these to update your systems (developer.paychangu.com).
- **Security**: Validate each webhook using a `Signature` header – compute SHA-256 HMAC of the payload using your *webhook secret* and compare (developer.paychangu.com).
- **Reliability**: Return HTTP 200 OK; PayChangu retries three times at 30-minute intervals if not acknowledged (developer.paychangu.com).
- Recommendation: always **re-verify** transactions via API before acting on them, especially for critical customer-facing operations .

---

4. Transaction Verification

After receiving a payment (via webhook or redirect), call the **transaction verification endpoint**:

GET https://api.paychangu.com/verify-payment/{tx_ref}

-
- Validate that the `tx_ref`, `status`, `amount`, and `currency` are as expected (developer.paychangu.com, developer.paychangu.com).
- Only then, **fulfill the order** or grant access.

---

5. Disbursements & Payouts

- Use similar API endpoints to send money (payouts) via mobile money or bank accounts.

- Triggering a payout returns a webhook (`api.payout`) when completed (m.facebook.com, developer.paychangu.com).

---

6. SDKs & Plugins

- Official SDKs available include:
    - **Python client** (`paychangu_py`) for transactions, payouts, airtime, with full features (developer.paychangu.com, github.com).
    - **Flutter/Dart library** (`paychangu_flutter`) with payment initiation & verification methods .
    - **Node.js**, **WooCommerce**, **GiveWP**, **WHMCS** plugins (developer.paychangu.com).

✅ Recommended Workflow

1. **Test your integration** using test credentials.
2. Choose your integration style: Popup, Link, Direct Charge.
3. Always enable **webhooks** and validate signatures.
4. **Verify transaction status** via API even after webhook fires.
5. Respond only after both webhook and verification confirm success.
6. **Handle failures gracefully** (e.g., allow retries, display clear user messages).

In Summary

PayChangu provides versatile payment tools—hosted or inline checkout, direct charges via mobile money/bank transfer, and send-out payouts. Security and reliability are top priorities: use signature verification, respond to webhooks quickly, and re-verify payment

details through the API before finalizing transactions. SDKs in major languages and plugin support for popular platforms make integration smooth.

Let me know if you want code samples (e.g. PHP, Node.js, Python) or help implementing a specific integration path!

Sure! Let's break down **Standard Checkout** integration with PayChangu step-by-step in simple terms:

🔁 What is Standard Checkout?

It's a **hosted payment link** system — your backend (server) creates a **payment URL** using PayChangu's API. You **redirect your customer** to that URL to complete the payment on PayChangu's secure page. When they finish, they are sent back to your site.

✅ Benefits:

- No frontend coding or UI needed.
- PayChangu handles all payment screens securely.
- Easier PCI compliance.

🛠️ How It Works

1. Your Server Calls PayChangu API

You send a **POST request** to create a payment.
Here's the API endpoint:

POST https://api.paychangu.com/v1/payments

Example Request Body:

```
{
  "amount": 5000,
  "currency": "MWK",
```

```
  "tx_ref": "ULIKUTI_001",
  "redirect_url": "https://yourapp.com/payment-success",
  "customer": {
    "email": "user@example.com",
    "phone_number": "0999123456",
    "name": "Alice Banda"
  },
  "customization": {
    "title": "Ulikuti Purchase",
    "description": "Payment for premium post"
  }
}
```

Required headers:

Authorization: Bearer YOUR_SECRET_KEY

Content-Type: application/json

---

2. PayChangu Responds with a Payment Link

You get a response like:

```
{
  "status": "success",
  "data": {
    "link": "https://pay.paychangu.com/checkout/TOKEN123"
  }
}
```

✅ Save the `tx_ref` for future verification!

---

3. Redirect Customer

On the frontend, redirect the user to that link:

<script>

  window.location.href = "https://pay.paychangu.com/checkout/TOKEN123";

</script>

---

4. Customer Pays on PayChangu

They see a hosted payment screen, pay using mobile money, card, etc.

---

5. User Redirected Back to Your Site

Once payment completes, they are sent to your `redirect_url`. It will include a `tx_ref` or transaction ID as a query param.

Example:

https://yourapp.com/payment-success?tx_ref=ULIKUTI_001

---

6. (Optional but Recommended): Verify Payment

Before giving the user access to a feature or order:

Call this endpoint:

GET https://api.paychangu.com/verify-payment/ULIKUTI_001

If status is `success`, then complete the order.

✅ Summary Workflow:

Your Server → PayChangu API → Redirect User → PayChangu Page → Redirect Back → Verify Payment → Grant Access

---

Would you like:

- A **working code example** (Node.js, Python, PHP)?
- Explanation of **verification**?
- Help with test mode setup?

Let me know!

Great! Let me explain **Inline Checkout** (via JavaScript SDK) step by step.

🧩 What is Inline Checkout?

This option uses **a popup** embedded on your website or app. Instead of sending the user away to a hosted page, PayChangu's checkout appears in a modal (popup) directly on your site.

It uses a JavaScript file called `popup.js`, and a function `PaychanguCheckout()` to trigger the payment.

✅ When to Use

- You want a **better user experience** — no page reload or redirection.
- You can work with **JavaScript** in your frontend (HTML, React, etc.).
- You want a **faster checkout flow**, similar to Paystack or Flutterwave popups.

🛠️ How to Implement It

1. Include the PayChangu JavaScript SDK

In your HTML file, add this:

<script src="https://checkout.paychangu.com/popup.js"></script>

---

2. Call the Checkout Function

You trigger the payment by calling `PaychanguCheckout()` with a config object:

<button onclick="makePayment()">Pay Now</button>

<script>

function makePayment() {

  PaychanguCheckout({

    public_key: "YOUR_PUBLIC_KEY",

    amount: 5000, // in MWK or selected currency

    currency: "MWK",

    tx_ref: "ULIKUTI_001", // your unique transaction ID

    redirect_url: "https://yourapp.com/payment-success",

    customer: {

      email: "user@example.com",

      phone_number: "0999123456",

      name: "Alice Banda"

    },

    customization: {

      title: "Ulikuti Checkout",

      description: "Pay for Ulikuti service"

    }

```
  });
}
</script>
```

🔄 What Happens After?

1. The **popup opens** on your site.
2. The user completes payment inside the popup.
3. They are **automatically redirected** to your `redirect_url`.
4. PayChangu triggers a **webhook** to your server (if configured).
5. The customer also receives a **payment receipt email**.

✅ Verification (Important!)

To protect against fake redirects, always **verify the payment** on your backend like this:

GET https://api.paychangu.com/verify-payment/ULIKUTI_001

Authorization: Bearer YOUR_SECRET_KEY

Only grant access if the status is `success`.

💡 Example Use Case

You're building a React or plain HTML/JS site and want a "Pay with PayChangu" button that pops up a fast, secure modal checkout. You don't want to send people away to a different website.

This is your best option.

📌 Summary

Step	Action
1.	Include `popup.js` in your HTML
2.	Trigger `PaychanguCheckout()` with payment config
3.	User pays in popup
4.	They're redirected to `redirect_url`
5.	You verify transaction on backend
6.	Fulfill order if payment is successful

Would you like:

- A **React version** of this example?
- Backend verification sample in Node.js or Python?
- Help setting up **webhooks** for your app?

Just ask!

This is an API & Webhook settings page for what appears to be a financial or payment service platform. Let me break down what each section does:

API Keys Section

Test Public Key: This is a public API key (starting with "PUB-TEST") that you can safely share with frontend applications or client-side code. Public keys are used to identify your account but can't perform sensitive operations.

Test Secret Key: This is shown as dots for security - it's your private key that should never be exposed publicly. Secret keys have full access to your account and can perform sensitive operations like processing payments or accessing private data.

The "Test" prefix indicates these are for development/testing environments, not live production use.

Webhook Setup Section

Webhooks allow the service to automatically notify your application when certain events occur (like a payment completion).

Test Webhook URL: This is where the service will send HTTP POST requests when events happen. You'd enter your server's endpoint URL here.

Test Webhook Secret: This is used to verify that webhook notifications are genuinely from this service. Your server should validate incoming webhooks using this secret to prevent spoofing.

Receive Webhook Notifications: This toggle appears to be currently disabled (the switch is off), meaning webhooks aren't being sent.

Purpose

This setup allows you to integrate your application with their service - the API keys let you make requests to their system, while webhooks let them push real-time notifications back to your application when important events occur.

The "Reset Keys" button would generate new API keys if your current ones were compromised.